

Fl.Shared.Utils.Components

Shared component library for .NET web applications. Provides reusable extensions, middleware and configurations for logging, exception handling, routing and service description.

Project structure

```
Fl.Shared.Utils.Components/  
├─ Constants.cs  
├─ Logging/  
│   ├── Models/  
│   │   └─ LoggingConfiguration.cs  
│   └─ Extensions/  
│       ├── Builder/  
│       ├── Either/  
│       ├── Error/  
│       ├── HostBuilder/  
│       └─ LoggerConfiguration/  
└─ Web/  
    ├── Endpoints/  
    ├── Extensions/  
    │   ├── ServiceCollection/  
    │   └─ WebApplication/  
    └─ Middlewares/  
        ├── ExceptionHandler/  
        ├── RequestLogContext/  
        ├── ServiceDescription/  
        └─ TraceLogContext/
```

Logging Models

SerilogConfiguration

Configuration record for Serilog, bindable from `appsettings.json`.

Property	Type	Default	Description
Environment	string	""	Environment name (e.g. <code>production</code>)
System	string	""	Application system name

Property	Type	Default	Description
Customer	string	""	Customer identifier
MinimumLevel	LogEventLevel	Information	Minimum log level
Overrides	IDictionary<string, LogEventLevel>	{}	Per-namespace log level overrides

Extensions

LoggerConfigurationExtensions

Extensions on `Serilog.LoggerConfiguration` to apply default settings.

```
// Configure with Information level and CompactJson console output
loggerConfig.ConfigureDefault();

// Configure with a specific level
loggerConfig.ConfigureDefault(LogEventLevel.Debug);

// Configure with a full SerilogConfiguration (also enriches with Environment,
// System, Customer)
loggerConfig.ConfigureDefault(serilogConfig);
```

HostBuilderExtensions

Extensions on `IHostBuilder` to add Serilog.

```
// From a SerilogConfiguration instance
hostBuilder.AddSerilogLogging(serilogConfig);

// From IConfiguration (binds SerilogConfiguration automatically)
hostBuilder.AddSerilogLogging(configuration);
```

LoggingWebApplicationBuilderExtensions

Extension on `WebApplicationBuilder` to add Serilog by reading configuration automatically.

```
builder.AddSerilogLogging();
```

EitherLoggingExtensions

Extensions on `Either<Error, T>` and `EitherAsync<Error, T>` to log the left (error) branch without breaking the functional chain.

```
// With an explicit template
either.TeeLog(logger, "{Component} returned {Message}", componentName, message);

// With only the component name (uses default template)
either.TeeLog(logger, "MyComponent");

// Also available for EitherAsync
eitherAsync.TeeLog(logger, "MyComponent");
```

ErrorExtensions

Extensions on `LanguageExt.Common.Error` to convert an error into a logging `Action`.

```
// With a custom template
var logAction = error.ToLoggingAction(logger, "{Component} error: {Message}",
componentName, error.Message);

// With component name only (uses default template "{Component} raised an error
with {Message}")
var logAction = error.ToLoggingAction(logger, "MyComponent");
```

Web Endpoints

IEndpoint

Interface for defining endpoints via minimal API.

```
public interface IEndpoint
{
    WebApplication DefineEndpoints(WebApplication app);
}
```

Implement this interface and register implementations via `AddEndpointDefinitions`.

Middleware

JsonExceptionHandlerMiddleware

Catches any unhandled exception in the pipeline and returns an HTTP 500 JSON response containing the error message.

```
{  
  "message": "Error description"  
}
```

Registration:

```
services.AddExceptionHandler();  
// ...  
app.UseJsonExceptionHandler();
```

RequestLogContextMiddleware

Enriches the Serilog log context with a `CorrelationId` property for every HTTP request, reading it from `ICorrelationContextAccessor`.

Registration:

```
services.AddRequestLogContext();  
// ...  
app.UseRequestLogContext();
```

TraceLogContextMiddleware

Enriches the Serilog log context with `SpanId` and `TraceId` properties from the current `Activity` (W3C trace context).

Registration:

```
services.AddTraceLogContext();  
// ...  
app.UseTraceLogContext();
```

ServiceDescriptionMiddleware

Exposes service metadata (name and version) at the `GET /internal/description` endpoint. All other requests are forwarded to the next middleware.

Example response:

```
{
    "name": "my-service",
    "version": "1.0.0"
}
```

Registration:

```
services.AddServiceDescription(configuration);
// ...
app.UseServiceDescription();
```

Configuration is loaded from `servicedescription.json` (or a custom file):

```
builder.AddServiceDescriptionConfiguration(); //
servicedescription.json (default)
builder.AddServiceDescriptionConfiguration("custom-file.json"); // custom file
```

IServiceCollection extensions

ServiceCollectionExtensions

```
// Registers all default components in a single call:
// - JsonExceptionHandlerMiddleware
// - RequestLogContextMiddleware
// - ServiceDescriptionMiddleware
// - CorrelationId
// - RoutingConfiguration
// - JsonStringEnumConverter
services.AddDefaultComponents(configuration);

// Registers a strongly-typed configuration object read from IConfiguration
services.AddFromAppConfiguration<MyConfig>(configuration);
```

RoutingBasePathServiceCollectionExtensions

Registers `RoutingConfiguration` to configure an optional base path.

```
services.AddRoutingBasePath(configuration);
```

JsonSerializationServiceCollectionExtensions

Adds `JsonStringEnumConverter` to the JSON serialization options.

```
services.AddJsonSerializationConfiguration();
```

EndpointServiceCollectionExtensions

Scans the specified assemblies and registers all `IEndpoint` implementations as singletons.

```
// From marker types
services.AddEndpointDefinitions(typeof(MyMarker));

// From assemblies
services.AddEndpointDefinitions(Assembly.GetExecutingAssembly());
```

WebApplication extensions

ApplicationBuilderExtensions

Registers the default middleware pipeline in the correct order.

```
app.UseDefaultMiddlewares();
// Equivalent to:
// app.UseServiceDescription()
// app.UseCorrelationId()
// app.UseRequestLogContext()
// app.UseJsonExceptionHandler()
```

RoutingBasePathWebApplicationExtensions

Configures the routing base path if `RoutingConfiguration.PathBase` is set.

```
app.UseRoutingBasePath();
```

EndpointWebApplicationExtensions

Calls `DefineEndpoints` on all `IEndpoint` implementations registered in the container.

```
app.UseEndpointDefinitions();
```

Tests

Unit tests are located in `tests/Fl.Shared.Utils.Components.Unit.Tests` and cover:

Area	Class under test	Scenarios
Web / Middleware	<code>JsonExceptionHandlerMiddleware</code>	Exception in delegate → 500 + JSON error message; no exception → next delegate invoked
Web / Middleware	<code>ServiceDescriptionMiddleware</code>	Request to <code>/internal/description</code> → 200 + JSON with name and version; other path → next delegate invoked
Logging / Extensions	<code>EitherLoggingExtensions</code>	Error logged on the left branch of <code>Either</code> and <code>EitherAsync</code>
Logging / Extensions	<code>ErrorExtensions</code>	<code>Error</code> converted to a logging <code>Action</code> , with and without an associated exception

Main dependencies

- [Serilog](#) — structured logging
- [LanguageExt](#) — functional types (`Either`, `Error`, etc.)
- [CorrelationId](#) — correlation ID propagation
- [Scrutor](#) — assembly scanning for `IEndpoint`
- `Fl.Functional.Utils` — functional utilities (Tee, Map, MakeOption, etc.)
- `Fl.Configuration.Extensions` — extensions for reading strongly-typed configurations

Namespace Fl.Shared.Utils.Components. Logging.Extensions.Builder Classes

[LoggingWebApplicationBuilderExtensions](#)